

DSA 8020 Statistical Methods II

Project III

Instructor: Whitney Huang (wkhuang@clemson.edu)

Due Date: May 1 (Thursday), 11:59pm ET

The ERA-Interim is a global atmospheric reanalysis dataset. Reanalysis refers to the process of producing spatially and temporally gridded datasets through data assimilation, primarily for climate monitoring and analysis. In the R data file `NA_MaxT_ERA.RData`, you will find daily maximum temperature values from 1979 to 2017, with a spatial resolution of approximately 80 km. The original global dataset has been subsetting to a region covering the contiguous United States and extending into Canada and Mexico.

In Problems 1 and 2, you are required to analyze an ERA-Interim average temperature time series at yearly and monthly temporal resolutions at a spatial location of your choice.

Problem 1 Analyzing Yearly Average Temperature Time Series

1. The following steps are needed to extract the annual mean temperatures at the specified location from the original dataset with daily temperatures at thousands of locations:

(a) First, use the following R script to load the R data object:

```
load("NA_MaxT_ERA.RData")
NA_MaxT <- NA_MaxT - 273.15 # change from Kelvin to Celsius
nlon <- length(NA_lon); nlat <- length(NA_lat)
nmon <- 12; nyr <- 39 # numbers of month/year
```

- (b) Second, choose a location of interest (with the information of longitude and latitude) and identify the nearest ERA grid cell to the chosen location. Below you can find a script to get the correct ERA grid cell for Clemson:

```
library(fields)
Clemson.lon.lat <- c(-82.8374, 34.6834)
# You can get lon/lat information from google
dist2Clemson <- rdist.earth(matrix(Clemson.lon.lat, 1, 2),
expand.grid(NA_lon, NA_lat), miles = F)
id <- which.min(dist2Clemson)
(lon_id <- id %% nlon)
(lat_id <- id %/% nlon + 1)
# Check
(rdist.earth(matrix(Clemson.lon.lat, 1, 2),
matrix(c(NA_lon[lon_id], NA_lat[lat_id]), 1, 2), miles = F)
== min(dist2Clemson))
```

- (c) Third, aggregate the daily data to the annual average time series. You could use the following script to achieve this:

```
ts_daily <- NA_MaxT[lon_id, lat_id,]
ts_yearly <- tapply(ts_daily, list(yr), mean)
```

2. Describe the main features of the annual time series data you extracted.
3. Is it reasonable to assume that there is a linear trend? If so, estimate and remove the trend to get the detrended time series.
4. Make `acf` and `pacf` plots for the detrended time series to identify possible ARMA models. That is, determine the possible orders p for the AR component and the orders q for the MA component.
5. Fit the possible ARMA models you identified and perform model selection to determine the ‘best’ model.
6. Perform a one-year-ahead forecast. Calculate the prediction and provide a 95%

prediction interval.

Problem 2 Analyzing Monthly Average Temperature Time Series

Repeat the analysis for Problem 1 but for the monthly average time series (which can be extracted using the script below). An important component of this problem is the analysis of seasonal patterns when dealing with sub-annual data. Some techniques from Week 14 should be employed to complete this problem.

```
ts_monthly <- tapply(ts_daily, list(mon, yr), mean)
```

Problem 3

In this problem we will use the `fields` package to conduct spatial interpolation using Gaussian process model. We use `ozone2` dataset in the `fields` package to interpolate average ozone levels.

1. Use the following script to load the data:

```
data(ozone2)
loc <- ozone2$lon.lat
rg <- apply(loc, 2, range)
y <- ozone2$y[16,]
good <- !is.na(y)
```

2. Visualize the spatial data using the script below and describe your findings:

```
library(maps)
map("state", xlim = rg[, 1], ylim = rg[, 2])
quilt.plot(loc[good,], y[good], nx = 60, ny = 48, add = T)
```

3. Make a variogram by assuming a linear spatial trend. You may use the following script to plot the variogram. Describe qualitatively what you learned about the spatial correlation from this variogram.

```
# remove linear spatial trend
lm <- lm(y[good] ~ loc[good,])

d <- rdist.earth(loc[good,]); maxd <- max(d)
vgram <- vgram(loc = loc[good,], y = lm$residuals, N = 30, lon.lat = TRUE)
plot(vgram$stats["mean",] ~ vgram$centers, main = 'Binned Semivariogram',
las = 1, ylab = "", xlab = "Dist (miles)", col = "red",
xlim = c(0, 0.3 * maxd), ylim = c(0, 700))
```

4. Use the following script to fit a Gaussian process to the data and to decompose the prediction into a spatial trend part and spatially correlated error part. Explain what mean function and covariance function are being used here.

```
fit <- spatialProcess(loc[good,], y[good])

out.full <- predictSurface(fit, extrap = T)
out.poly <- predictSurface(fit, just.fixed = T, extrap = T)

out.spatial <- out.full
```

```
out.spatial$z <- out.full$z - out.poly$z

set.panel(1, 3)
surface(out.full, las = 1, xlab = "Lon", ylab = "Lat", col = tim.colors())
title("Full model")
map("state", add = T)
surface(out.poly, las = 1, xlab = "Lon", ylab = "Lat", col = tim.colors())
map("state", add = T)
title("Spatial trend")
surface(out.spatial, las = 1, xlab = "Lon", ylab = "Lat", col = tim.colors())
map("state", add = T)
title("Spatial error")
```

5. Make prediction map and the associated prediction uncertainty map using the script below. Can you explain the spatial variation in the uncertainty map?

```
xg <- fields.x.to.grid(loc[good,])
Pred <- predict.mKrig(fit, xnew = expand.grid(xg))
SE <- predictSE(fit, xnew = expand.grid(xg))

par(mfrow = c(1, 2), las = 1)
map("state", xlim = rg[, 1], ylim = rg[, 2])
image.plot(xg[[1]], xg[[2]], matrix(Pred, 80), add = T)
map("state", xlim = rg[, 1], ylim = rg[, 2], add = T)
title("Prediction")
map("state", xlim = rg[, 1], ylim = rg[, 2])
image.plot(xg[[1]], xg[[2]], matrix(SE, 80), add = T)
map("state", xlim = rg[, 1], ylim = rg[, 2], add = T)
title("Prediction SE")
points(loc, pch = 16, cex = 0.5)
```